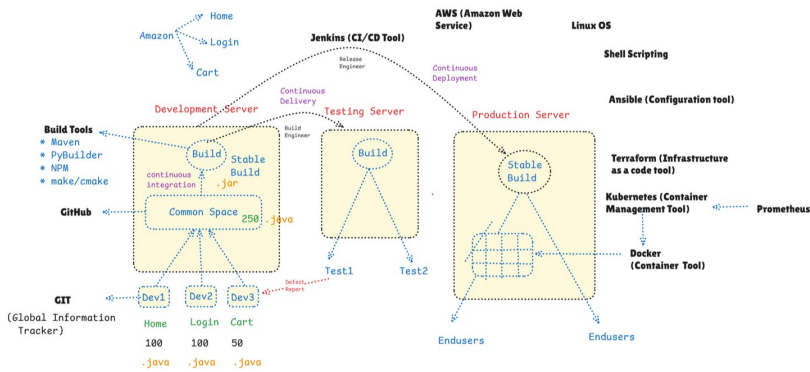


GIT (Global Information Tracker)



Drawbacks

- > No Collaboration between the developers
- > No Backtracking
- > It is a time consuming process
- > Dataloss

VCS (Version Control System)

- > It is used to manage and track the versions of the source code



- * Versions: Each time the file being saved will be treated as a version

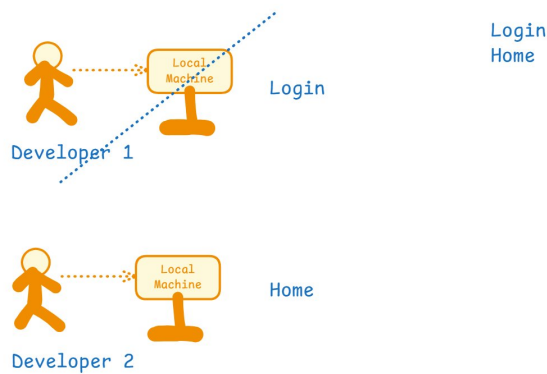
Track:

WHO have made the changes
WHAT changes have been made
WHEN the changes were made

There are different types of version control system (VCS)

1. Localized/local version control system
2. Centralized/Central version control system
3. Distributed Version Control System

1. Localized/Local Version Control System



Advantages

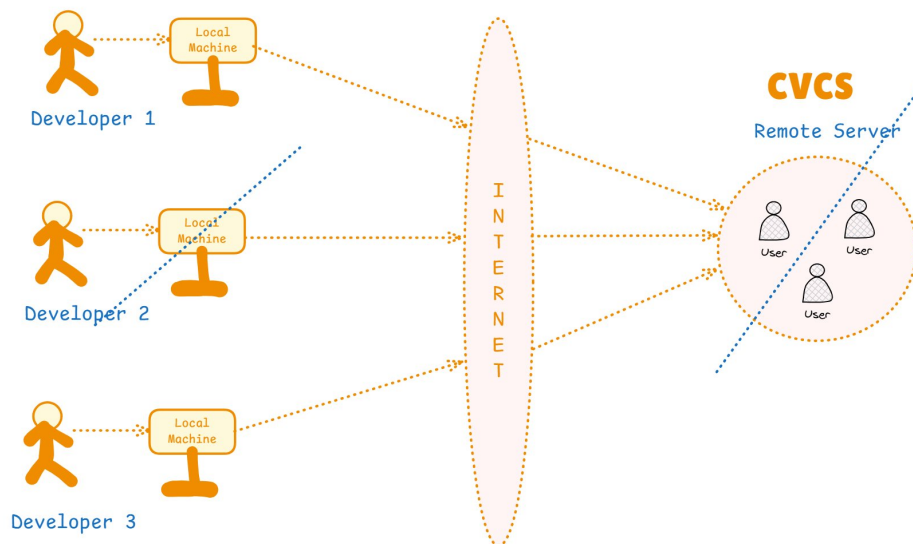
- ```
--> The functionality itself we are going to take it as an
 advantage
--> By using localized version control system you will be
 able to track the versions of the source code
```

## Dissadvantages

- > No collaboration between the developers
- > Risk of losing the data is high in case of machine failure
- > No backup

Eg: SCCS (Source code control system)  
RCS (Revision version control system)

## 2. Centralized Version Control System



### Advantages

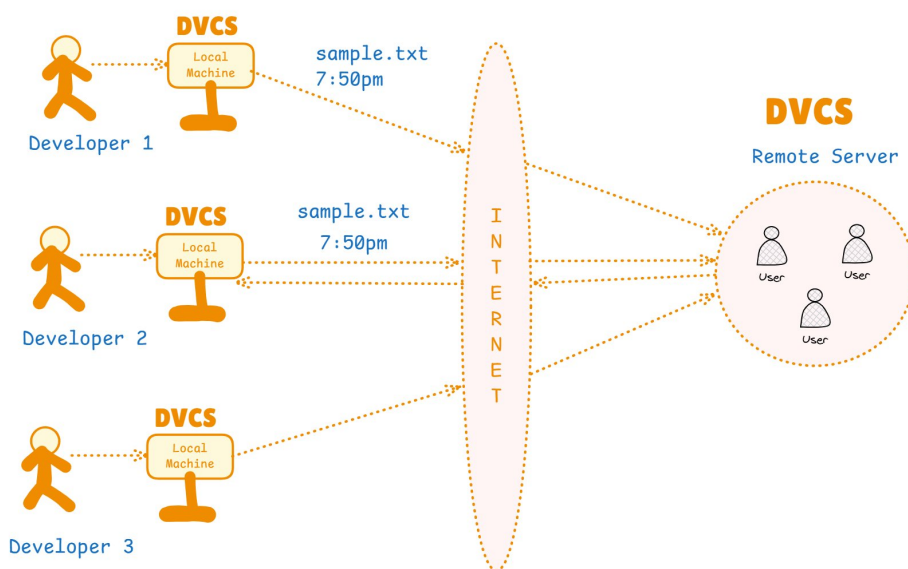
- > You will be track the versions of the source code
- > The Collaboration is possible

### Dissadbatages

- > Internet is mandatory
- > Single point of failure --> (If server goes down no one can save the changes in the remote server)
- > Risk of loosing is high

Eg: Subversion (SVN), Perforce

## 3. Distributed Version Control System (DVCS)



### Advantages

- > We can easily collaborate with other developers
- > Internet is not mandatory
- > Risk of loosing data is less compared to other version control system
- > By using DVCS you will be able track the versions of the source

Eg: GIT, Mercurial, Bazaar

CVCS: Everyone shares one copy of the project on a server, you must be online to save changes

DVCS: Everyone has their "own full copy". You can make changes offline and later sync with other developers

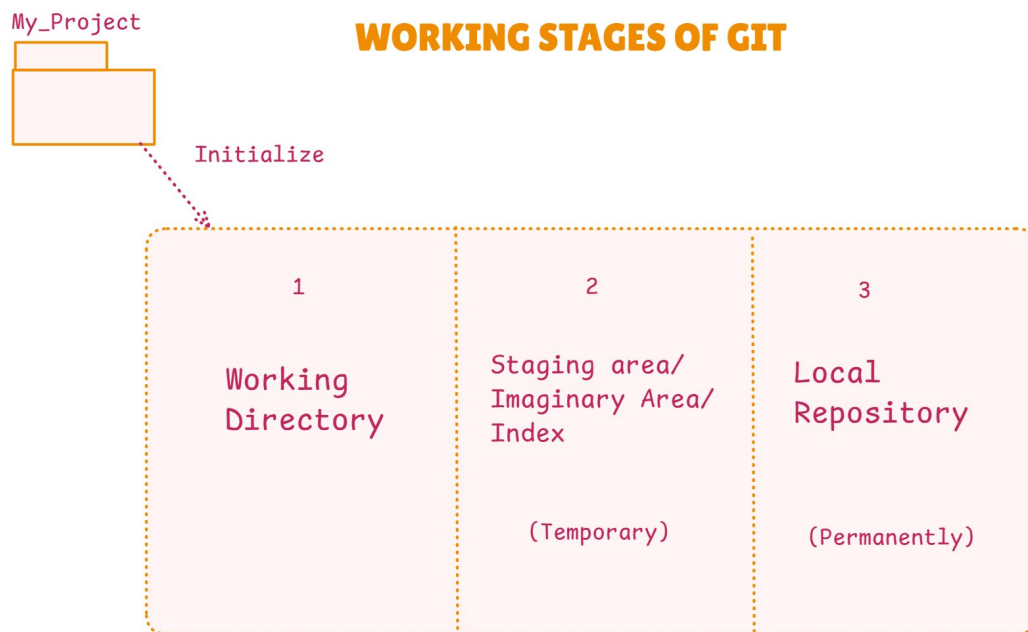
# History of GIT

- > Linus Torvalds was using a tool called "Bitkeeper" which was used for managing, tracking, and keeping the backup
- > Bitkeeper tool was licensed that means it was not free and open source to all the users

2005: Linus Torvalds introduced a tool that is GIT, which was free and opensource to all the users

## GIT (Global Information Tracker)

- > GIT stands for Global Information Tracker
- > It is a version control tool which follows distributed version control system
- > GIT is used to track and manage the changes of the source code
- > GIT is also called as source code management tool
- > GIT you will be track the following details
  1. WHEN : When the changes were made
  2. WHO : Who have made the changes
  3. WHAT : What changes where made



--> Whenever we start using git, we need to first initialize at the location where we want to use git

--> After initialization we will be getting an hidden folder that is ".git"

--> Once after this, the folder will be segregated into 3 different sections

### 1. Working Directory

It is a place where we will be creating the files or performing any read/write operations

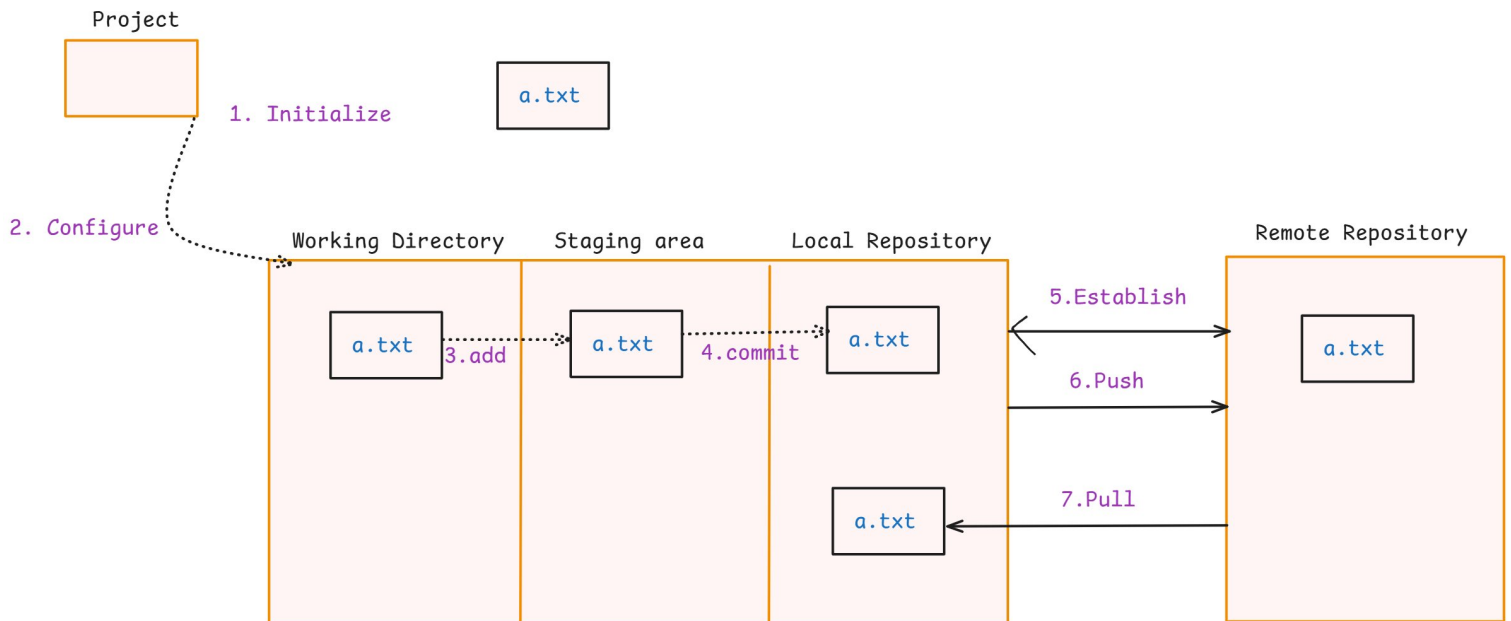
### 2. Staging area

This is the place where we are going to keep the files for the temporary period of time

### 3. Local Repository

This is the place where we are going to save the versions of the source code permanently in the local machine

# WORKFLOW OF GIT



## 1. Initialization

Before using GIT we need to initialize the location/directory

```
git init
```

## 2. Configuration

These will be used in order to know "WHO" have made the changes

```
Username: git config user.name "username"
```

```
User-email: git config user.email "xyz@gmail.com"
```

--> If you want to check the username and user email

```
git config --list
```

--> If you want to check only the configured user name

```
git config user.name
```

--> If you want to check only the configured user email

```
git config user.email
```

## 3. Add

To move the files from the working directory to the staging area is called as adding

--> `git add file_name`: Only the specified file will be added to the staging area

--> `git add file1 file2 file3`: All the specified files in the command will be added to the staging area

--> `git add .`: All the files from the current working directory will be added to the staging area

--> `git add *.extension`: All the files with the specified extensions will be added to the staging area

>> To check the files whether it is present in the working directory or the staging area we have a command that is

```
git status
```

git status

#### 4. Commit

To move the files from the staging area to the local repository is called as "Committing"

--> git commit -m "added my first project"

--> To check the commit history

--> git log                      OR              git log --oneline

#### 5. Establishing the connection

git remote add {alias\_name} remote-URL

↓  
origin

--> To verify the connection

git remote -v

--> To remove the connection

git remote remove {alias\_name}

↓  
origin

#### 6. Push

Uploading the files from the local repository to the remote server

git push origin master/main              OR      git push -u origin master

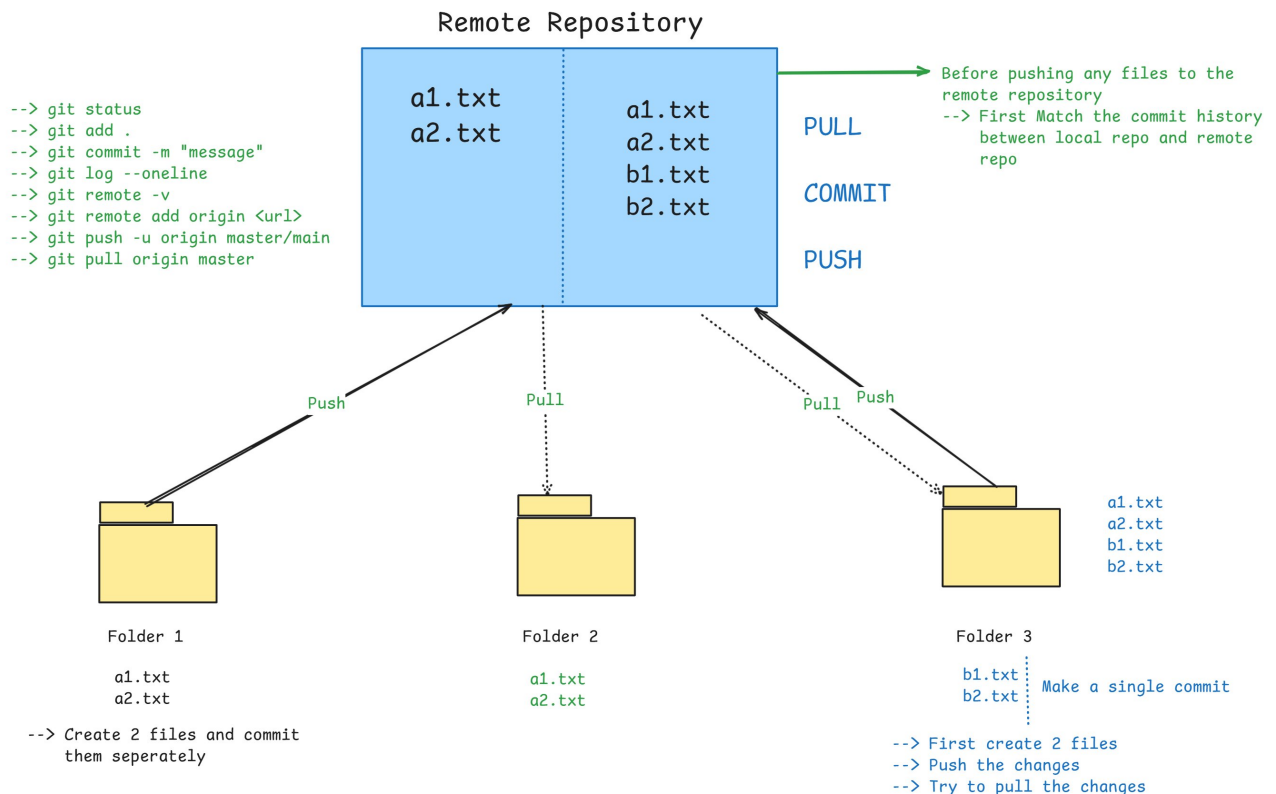
#### 7. Pull

To get the changes from the remote repository to the local repository

git pull origin master/main

--> If the file are in red color that means those files are being untracked and present in the working directory

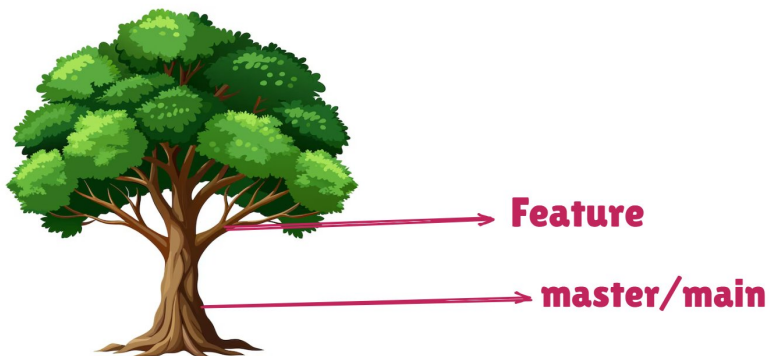
--> If the files are present in the green color that means currently those files are being tracked in the staging area



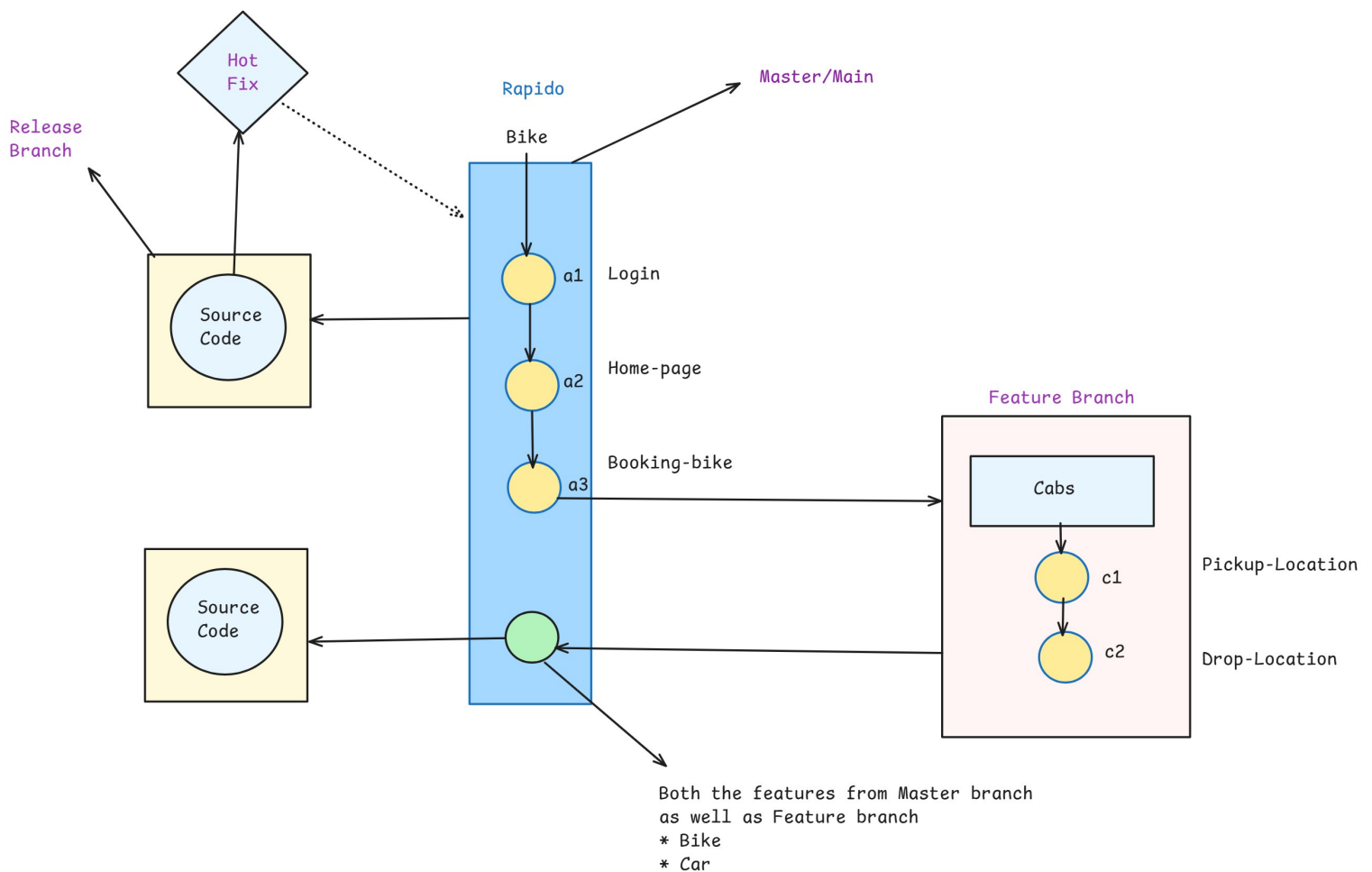
--> Inorder to push and pull the further changes, we need to make sure the commit history is matching

git push origin master/main --force (To push the files from local repository to remote repository forcefully)  
It will delete the contents which is already present in the remote repository

# Branching



1. Master/Main branch  
It is a default branch where you initially write the code  
--> It always contains production ready code
2. Feature Branch  
--> From this the new features will be introduced to the applications
3. Release branch  
--> It is a branch used for production release
4. Hotfix branch  
--> Resolving the immediate issues (hotfix) of the application is done here



# Commands used in Branching

1. To list the branches present locally

```
git branch OR git branch --list
```

2. To create a branch

```
git branch branchname
```

3. To switch the branches

```
git switch branch_name OR git checkout branch_name
```

4. To create and switch to the branch simultaneously

```
git checkout -b branch_name
```

5. To push the branches

```
git push origin branch_name
```

6. To list the branches which is present in the remote server

```
git branch -r
```

7. To list the branches which is present locally as well as in the remote server

```
git branch -a
```

8. To pull the branches

```
git pull origin branch_name
```

9. To delete the branches present locally

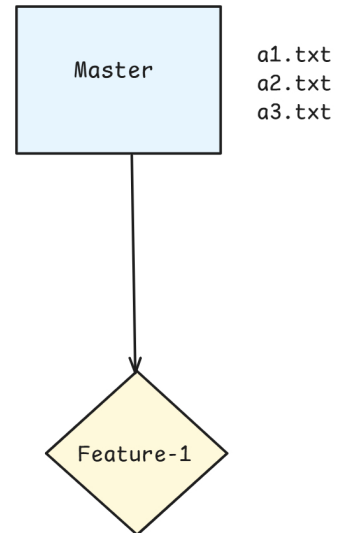
```
git branch -d branch-name OR git branch -D branch-name
 (To delete the branches forcefully)
```

10. To rename the branches

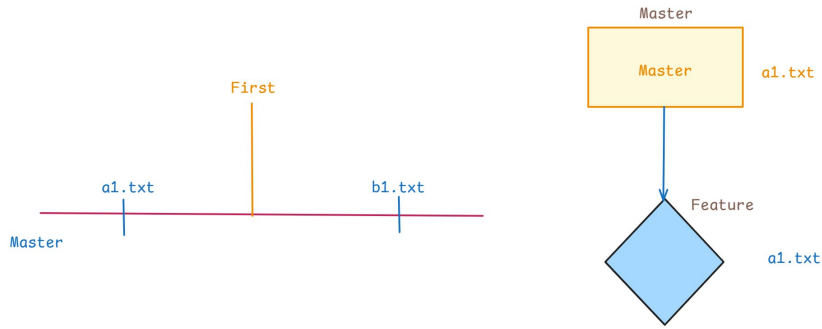
```
git branch -m branch-name (To rename the branch if your currently working on that)
git branch -m <old-branch-name> <new-branch-name>
```

11. To delete the branch from the remote repository

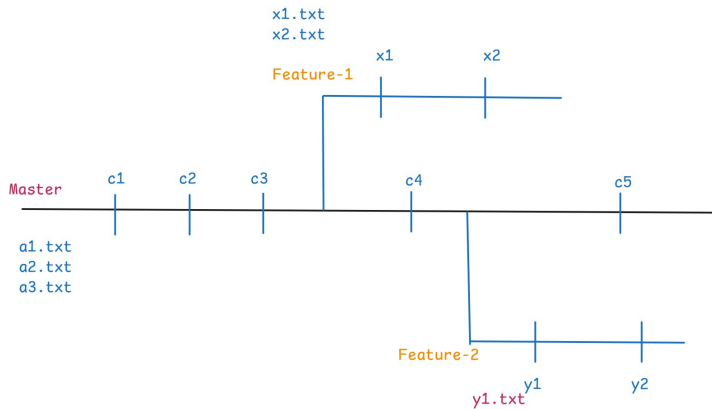
```
git push -d origin branch-name
```



## TASK-1

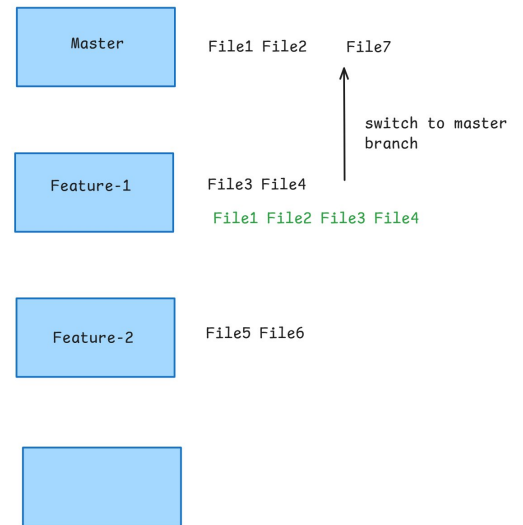


## TASK-2



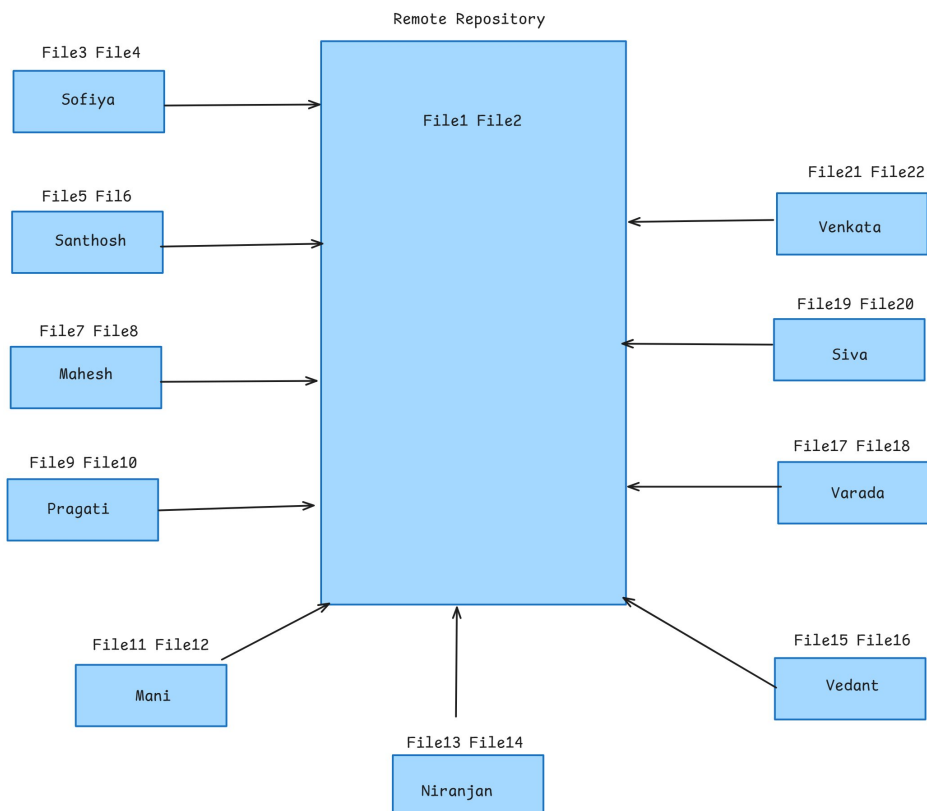
For c4: Add the content inside a1.txt  
 For y1: Add empty y1.txt  
 For y2: Add content inside y1.txt  
 For c5: Add content a2.txt as well as a3.txt

--> Check the commit history for all the branches  
 --> Push only Feature-1 branch to the remote repository  
 --> List the branches which is present both locally and in the remote server



## Collaboration

Collaboration in GIT refers to how multiple people work together in a code base using GIT



# Merging

Combining the changes from one branch to another branch is called as merging

Different types of Merging

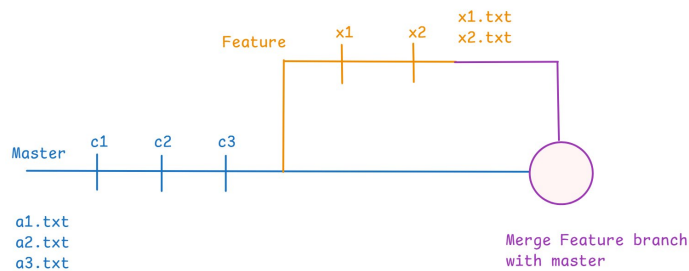
1. 2-way merging (Fast-Forward merging)
2. 3-way merging

Command for merging

```
git merge <branchname>
```

## \* Two way merging

--> It is also called as fast forward merging

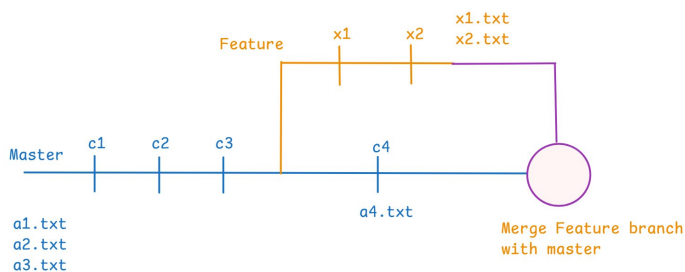


--> In this we will be creating some commits in the master branch and then create a feature branch and do some commits in the feature branch and later merging feature branch with the master branch is called as two way merging.

Note: While passing the merge command you have to be careful by being in which branch you are going to pass the commands

While merging make sure to be present in the branch in which you need to merge with another branch

## \* Three way merging

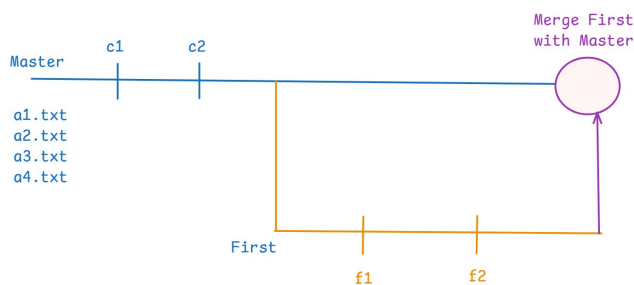


--> In three way merging, we are going to make the initial commits in the master branch, then create a feature branch and make some commits in the feature branch, later again switch to master branch and do one more in the master

--> Now when you try merge Feature branch with the master branch we will be getting an extra commit.. This process we call it as three way merging

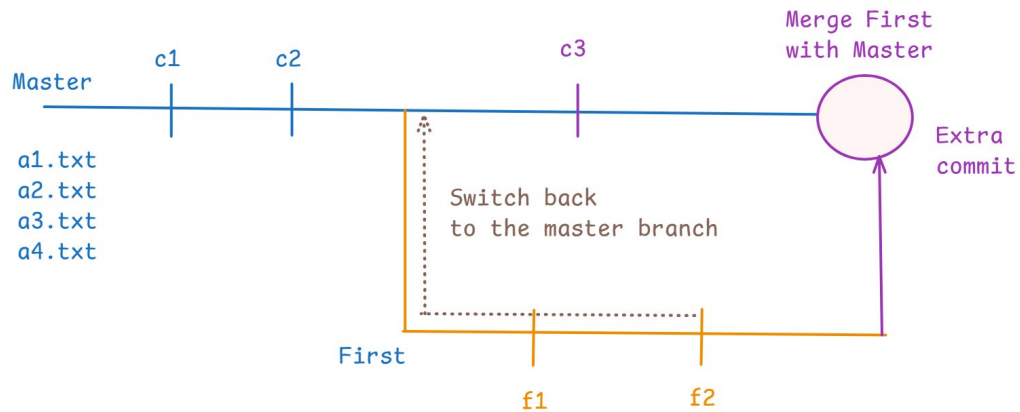
\* Three way merging is also called Optimized recursive three way merging

## TASK on Two way Merging



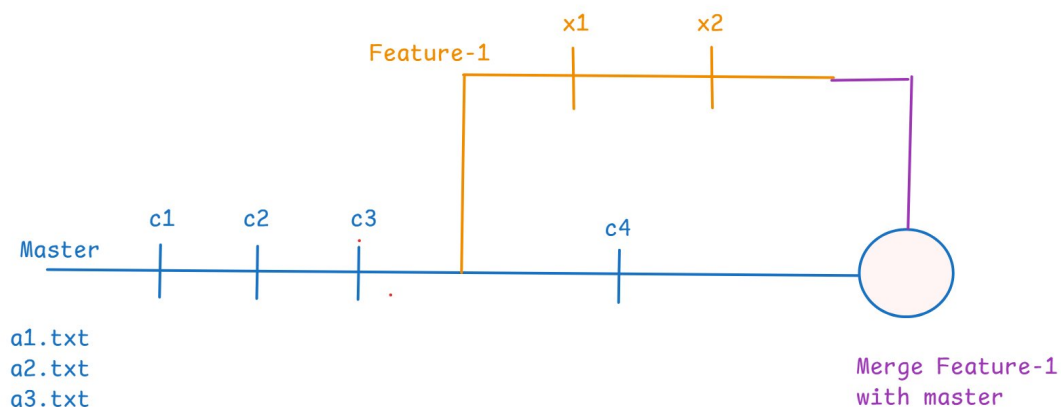
--> c1: Add all the empty files  
--> c2: Modify a2.txt  
--> f1: Modify a1.txt and a3.txt  
--> f2: Modify a4

## TASK on Three way Merging



--> c1: Add all the empty files  
--> c2: Modify a1  
--> f1: Modify a2 and a3  
--> f2: Modify a4  
--> c3: Again modify a1

## Merge Conflict



--> c1: Add all the empty files  
--> c2: Add the content inside a1.txt  
--> c3: Add content in a2.txt  
--> x1: Add content in a3.txt  
--> x2: Add empty x1 and x2 files  
--> c4: Add content in a3.txt

--> When we are modifying same file in both the branches, and if we try to merge the branch, we will get an error or conflict this process is called as Merge Conflict  
--> Because the GIT will get confused to take changes from which branch

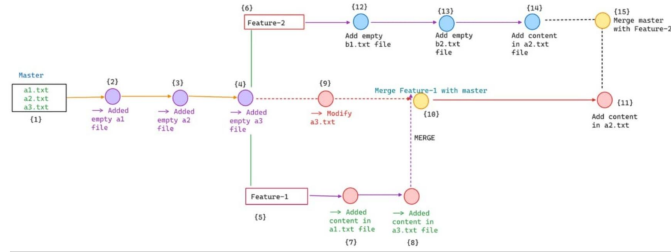
### How to resolve merge conflict

--> We need to manually edit the content of the file which is causing conflict and save it by removing the extra markers  
--> We need to add the file to the staging area and then commit the file

To add and commit a modified file simultaneously

```
git commit -am "msg"
```

## TASK-1



## TASK-2

1. Create a folder with 2 txt files and push that into master branch
2. Create "first" branch, and add 2 more text files and push into "first" branch
3. Create "second" branch from "first" branch and modify all the files
4. Modify all the files present in the "master" branch
5. Modify all the files present in the "first" branch
6. Merge first with master branch
7. Merge second with master branch
8. Create a folder with 2 txt files and push it into third branch
9. Create a fourth branch, add 2 more text files and push to the fourth branch
10. Merge third with first branch
11. Merge fourth with second branch
12. Push all the files and branches to the remote repository

## Undoing Changes

--> Reverse the doing

Undoing the changes can be done in three different ways

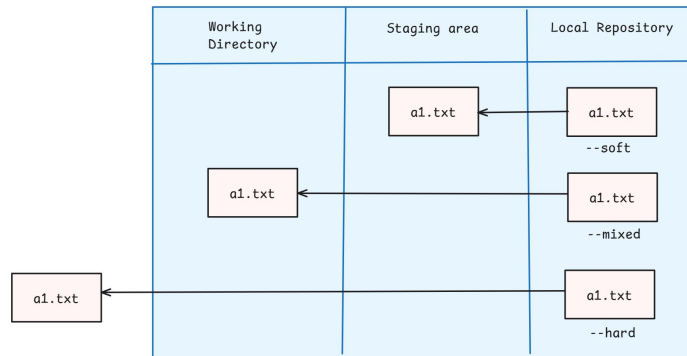
1. Reset
2. Checkout
3. Revert

### \* Reset

In Reset we have 3 different modes

1. Soft
2. Mixed
3. Hard

a1.txt



--> Undo commits or unstage files

1. --soft

```
git reset --soft <commit_id>
OR
git reset --soft HEAD~1
```

Undoing the changes by sending the file from local repository back to the staging area is called as soft

2. --mixed

```
git reset --mixed <commit_id>
OR
git reset --mixed HEAD~1
```

Undoing the changes by sending the file from local repository back to the working directory is called as mixed

3. --hard

```
git reset --hard <commit_id>
OR
git reset --hard HEAD~1
```

Undoing the changes by directly deleting the files from the git initialization folder is called as hard

## 2. Checkout

It will undo the modification or changes which are made in the working directory

--> Checkout will match the content of the file with the last commit which was made

--> Undo the modification for a particular file

```
git checkout <filename>
```

--> Undo the modification for all the files which is present in the working directory

```
git checkout .
```

--> The alternative command for checkout is

```
git restore <file_name> OR git restore .
```

## 3. Revert

--> Revert is also used for undoing the changes

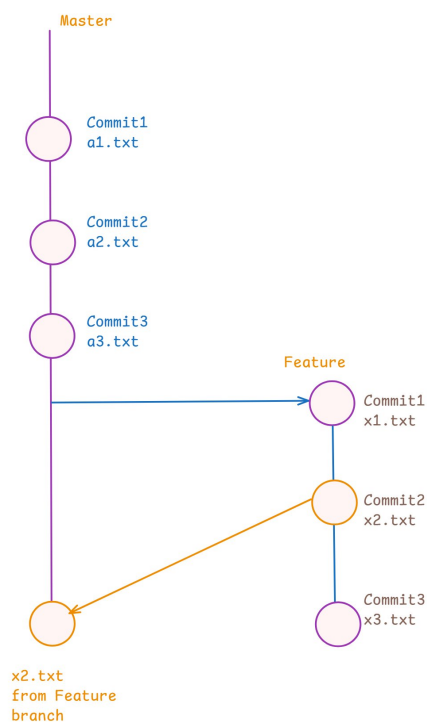
--> Revert will undo the commits and changes associated to the commits but it is not going to remove the deleted commit message from the commit history

```
git revert <commit_id>
```

## --> Cherry Pick

The cherry pick command is used to specify commit from one branch to another

```
git cherry-pick <commit-id>
```

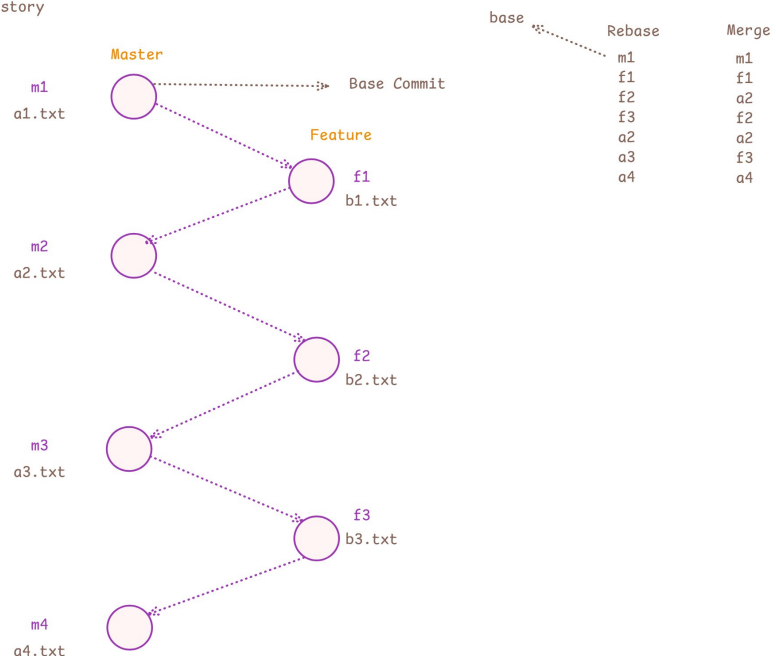


## --> Rebase

It is used to rewrite the commit history

```
git rebase <Feature> <master>
```

Rebase the commits from "Feature" branch to the "Master" branch



# Additional Commands

## **FORKING**

To get a copy of repository from one github account to another github account

**git config --global user.name "username"**  
**git config --global user.email "useremail"**

To globally configure the username and useremail

**git config --global --unset user.name**  
**git config --global --unset user.email**

To remove the globally configured username and useremail

## **git commit --amend**

This is the command used to edit the latest commit message

## **git config --global --edit**

Command to edit the globally configured username and user email

## **git restore --staged <file-name>**

Command to move the file from the staging area to the working directory

## **vi .gitignore**

Command to ignore any files without staging or commitng

## **cat .gitignore**

To see the files which are ignored from the working directory

## **git tag v1.1**

--> Command to assign a tag  
--> It assigns the tag to the latest commit message

## **git tag v1.2 <commit-id>**

Command to assign the tags based on the commit id

## **git push origin --tags**

Command to push the tags from local repository to the remote repository

## **git tag -l**

Command to list the tags

## **git tag -d <tag-name>**

Command to delete the tag in the local repository

## **git push origin --delete <tag-name>**

Command to delete the tag in the remote server (GitHub)

## **git stash**

It will save the changes but it not applicable to commit

## **git stash list**

To get the stash list

## **git stash apply stash@{0}**

Command to get the content back after stash

## **git stash drop stash@{0}**

Command to delete the content from the stash

## **git stash push -m "message" <file-name>**

Command to stash a specific file

## **git remote remove origin**

Command to remove the established connection between local repository and the remote server

## **git clone <github-url>**

Command to download the file from remote repository to the local machine